

คู่มือการใช้งานและพัฒนาระบบบริการ
ระบบเว็บเซอร์วิสบนระบบฝังตัว

User Manual & Platform Developer
Embedded Web Service Development
Embedded SOAP Server

โดย

นายชเนต ชูวิเศษวณิชย์ 43010175

นายธีรยุทธ สุทธิสุวรรณ 43010189

อาจารย์ที่ปรึกษา

ผศ. อภิเนตร อุณาภูณ

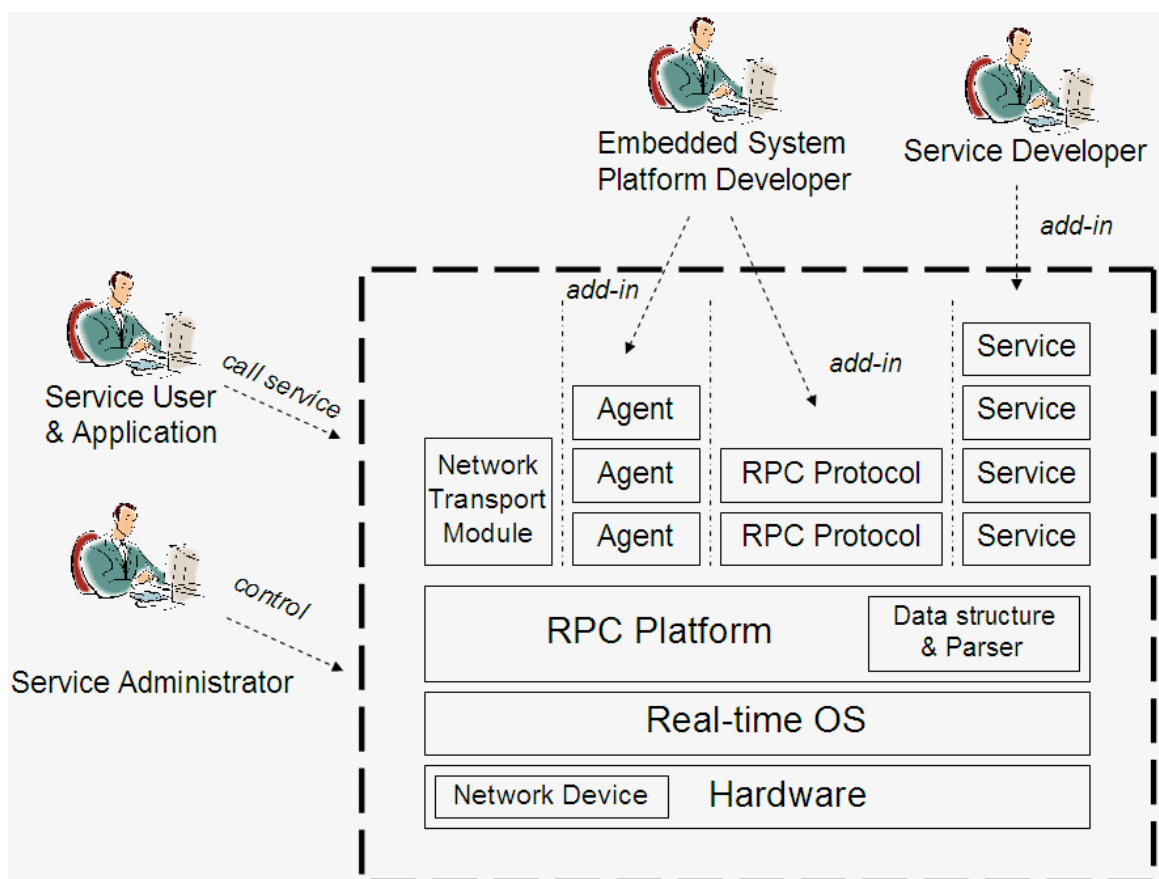
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ปีการศึกษา 2546

คู่มือการพัฒนาเว็บเซอร์วิสบนอุปกรณ์ฝังตัว

Platform Developer Manual

ลักษณะการใช้งานเว็บเซอร์วิสบนระบบฝังตัวนั้นสามารถแบ่งได้ออกเป็น 4 ลักษณะ คือ

1. นักพัฒนา
2. นักพัฒนาบริการบนอุปกรณ์ฝังตัว
3. ผู้ที่เรียกใช้บริการของระบบ
4. ผู้ดูแลระบบการให้บริการของระบบ



รูปภาพ 4-1: บทบาทของผู้ที่เกี่ยวข้องกับระบบ

4.1 นักพัฒนา

เป็นผู้ที่นำโค้ดของระบบ ไปพัฒนาในส่วนต่างๆต่อไปใช้ในงานเฉพาะอย่างที่ต้องการ โดยสามารถเพิ่มส่วนต่างๆ ดังนี้

- โปรโตคอลแฮนเดิลเลอร์ (Protocol Handler)

เป็นส่วนที่รองรับการติดต่อในรูปแบบของโปรโตคอลในมาตรฐานต่างๆ โดยส่วนนี้ ได้ถูกออกแบบให้สามารถขยายและเพิ่มเติมโปรโตคอลแฮนเดิลเลอร์ได้โดยนักพัฒนา โดยมีรูปแบบดังนี้

Protocol Handler In Task()

```
{  
RPCParam *param;  
MessageQ *mq;  
    for(;;){  
        //wait for request  
        //handle protocol to parameterobject  
        //save in parameter object  
        //save session infomation in RPCobject  
        //post return to requested task  
    }  
}
```

รูปภาพ 4-2: โครงสร้างการพัฒนาโปรโตคอลแฮนเดิลเลอร์ในส่วนของการดึงข้อมูลมาสู่อาร์พีซีพารามิเตอร์

จากรูปโครงสร้างของโปรแกรมในส่วนของโปรโตคอลแฮนเดิลเลอร์นี้ เป็นส่วนที่ทำหน้าที่อ่านข้อมูลจากมาตรฐานโปรโตคอล แล้วสร้างเก็บอาร์พีซีพารามิเตอร์เพื่อเก็บข้อมูลที่อ่านได้ แล้วเก็บบันทึกไว้ในพารามิเตอร์ออบเจกต์นั้น แล้วทำการเก็บบันทึกค่าที่จำเป็นต้องใช้ในการติดต่อในเซสชันของโปรโตคอลนั้นเอาไว้ในอาร์พีซีออบเจกต์ เมื่อทำงานเสร็จแล้ว ก็ทำการส่งแมจเซกไปยังทาสก์ที่เรียกใช้เพื่อให้ทำงานในลำดับอื่นๆต่อไป

Protocol Handler Out Task()

```
{  
  RPCParam *result;  
  MessageQ *mq;  
  for(;;){  
    //wait for request  
    //encode and serial data to message protocol  
    //post return to requested task  
  }  
}
```

รูปภาพ 4-3: โครงสร้างของโปรโตคอลแฮนเดิลเลอร์สำหรับแปลงข้อมูลกลับไปเป็นเอกสารของโปรโตคอล

จากรูป เป็นโครงสร้างของโปรโตคอลแฮนเดิลเลอร์ที่ทำหน้าที่แปลงข้อมูลที่เป็นผลลัพธ์ของการเรียกใช้บริการ ให้อยู่ในรูปของเอกสารตามมาตรฐานของแต่ละโปรโตคอล แล้วส่งผลกลับไปยังทาสก์ที่ร้องขอต่อไป

เนื่องจากระบบอาร์พีซีนี้ มีรากฐานมาจากแนวความคิดของสถาปัตยกรรมการทำงานแบบเว็บเซอร์วิส และเป็นโครงการที่พัฒนาต่อ จึงมีการดัดแปลงโซบโปรโตคอลแฮนเดิลเลอร์ มาเป็นโปรโตคอลแฮนเดิลเลอร์ที่เตรียมไว้ให้ในระบบแล้ว เพื่อสามารถนำไปใช้ทำการสื่อสารแบบเว็บเซอร์วิสได้

- เซิร์ฟเวอร์แดมอนทาสก์ (Server Daemon Task)

เป็นส่วนที่คอยรับการขอทำการเชื่อมต่อกับผู้ร้องขอบริการ ซึ่งต้องรูปแบบในการติดต่อในข้อมูลในระดับของเซสชัน (Session Layer) โดยรองรับโปรโตคอลที่ทำหน้าที่เป็นผู้รับส่งข้อมูล (Agent) ซึ่งได้แก่ โปรโตคอลเอชทีทีพี โปรโตคอลเอฟทีพี หรือ โปรโตคอลเอสเอ็มทีพี เป็นต้น ในการพัฒนาเซิร์ฟเวอร์แดมอนนั้น จะควรจะพัฒนาตามรูปแบบของโครงสร้างที่ได้ออกแบบไว้ตามหน้าที่การทำงานของเซิร์ฟเวอร์แดมอน ดังนี้

```

<serverd>_init();

<serverd>_thread();

<serverd>_close();

do_read();

```

รูปภาพ 4-4: โครงสร้างฟังก์ชันภายใน ในการอิมพลีเมนต์เซิร์ฟเวอร์เดมอน

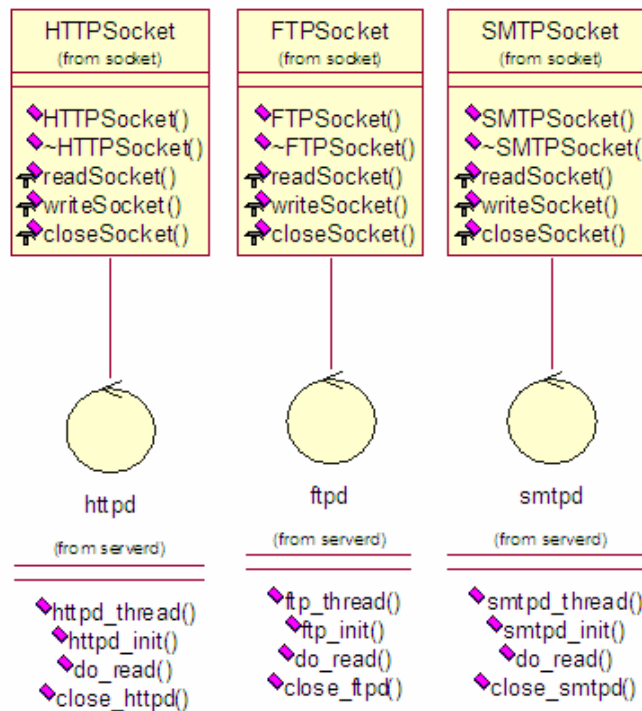
(*หมายเหตุ : <serverd> คือ ชื่อของโปรโตคอลใดๆ ที่จะถูกอิมพลีเมนต์เข้ามาใช้ในระบบ)

ตาราง 4-1: อธิบายฟังก์ชันในการอิมพลีเมนต์เซิร์ฟเวอร์เดมอน

<serverd>_init()	เป็นฟังก์ชันที่เริ่มการทำงานของระบบ โดยจะเรียกฟังก์ชันของระบบที่ทำหน้าที่สร้างเทรด (thread) ของเซิร์ฟเวอร์เดมอน เพื่อรอรับการร้องขอเชื่อมต่อในโปรโตคอลที่เซิร์ฟเวอร์เดมอนนั้นๆ รองรับ
<serverd>_thread()	เป็นฟังก์ชันที่เป็นส่วนที่ประมวลผลของเทรดของเซิร์ฟเวอร์เดมอน มีลักษณะคล้ายทาสก์ ที่วนรอบการทำงานตลอดเวลา
<serverd>_close()	เป็นฟังก์ชันที่ปิดการเชื่อมต่อในเซสชันนั้นๆ
do_read()	เป็นฟังก์ชันในการอ่านค่าข้อมูลเข้ามาในเซิร์ฟเวอร์เดมอนแล้วเก็บลงในโครงสร้างข้อมูลที่เตรียมไว้

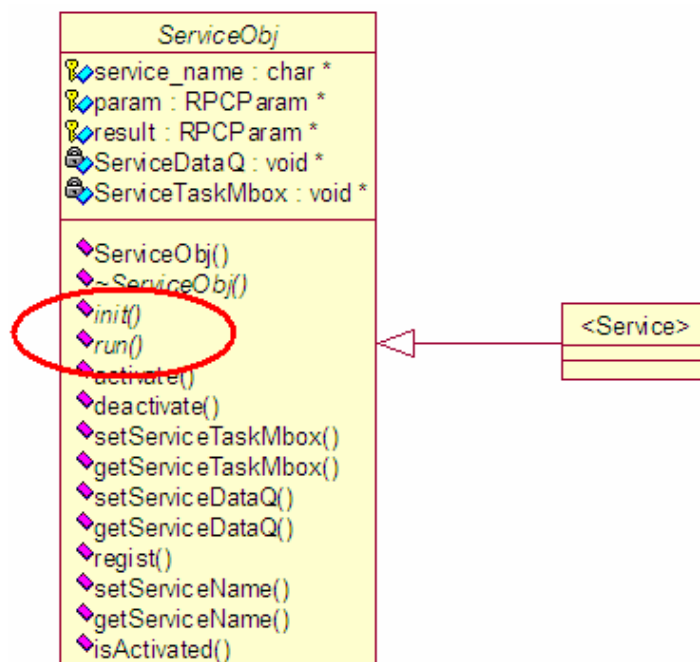
- **โปรโตคอลซ็อกเก็ต (Protocol Socket)**

เป็นคลาสที่จัดการการเชื่อมต่อของต่อละเซสชัน โดยสร้างขึ้นมาเพื่อรองรับรูปแบบเซสชันที่หลากหลายตามแต่โปรโตคอล เพื่อให้ทำงานร่วมกับทาสก์และเซิร์ฟเวอร์เดมอนได้อย่างถูกต้อง



รูปภาพ 4-5: ตัวอย่างการอิมพลิเมนต์ซ็อกเก็ต เพื่อทำงานร่วมกับเซิร์ฟเวอร์เดมอนในแต่ละโปรโตคอล

4.2 นักพัฒนาบริการบนอุปกรณ์ฝังตัว



รูปภาพ 4-6: การสร้างบริการ โดยการอิมพลิเมนต์เซอร์วิสออบเจกต์

ในการสร้างบริการหรือเซอร์วิศโค้ด เข้ามาใช้ในระบบนั้น นักพัฒนาจะต้องทำการอิมพลิเมนต์คลาสของเซอร์วิสออบเจกต์ ที่มีอยู่ในระบบ เพื่อใช้อินเตอร์เฟส (Interface) ที่จะเข้าไปทำงานร่วมกับระบบได้อย่างถูกต้อง โดยส่วนที่ต้องอิมพลิเมนต์เพิ่มนั้นมีดังนี้

- ฟังก์ชัน **init()**

เป็นฟังก์ชันที่ทำหน้าที่ในการแปลงข้อมูลจากพารามิเตอร์ที่ส่งเข้ามา ในรูปของสตริง (String) ไปเป็นข้อมูลในชนิดที่ระบบต้องการใช้ในการประมวลผล เพื่อเตรียมความพร้อมก่อนที่จะทำการประมวลผลต่อไป

- ฟังก์ชัน **run()**

เป็นฟังก์ชันหลัก ที่เป็นส่วนประมวลผลของบริการหรือเซอร์วิสนั้นๆ ในลักษณะของการทำงานแบบฟังก์ชันทั่วๆ ไป ที่ง่ายต่อการสร้างและพัฒนา โดยไม่ต้องสนใจส่วนอื่นๆ ในระบบ เพียงแต่เมื่อทำการประมวลผลฟังก์ชันเสร็จแล้ว ก็ให้แปลงข้อมูลจากชนิดที่เป็นอยู่ ไปเป็นสตริง แล้วนำไปเก็บไว้ที่อาร์พีซีรีซัลต์ เพื่อรอการนำไปจัดส่งผลลัพธ์กลับไปยังผู้ร้องขอตามโปรโตคอลต่างๆ ต่อไป

```
#ifndef CALCULATOR
#define CALCULATOR
#include "servobj.h"

class Calculator : public ServiceObj{ //extend and implement ServiceObject
public:

    Calculator(const char* serv_name):ServiceObj(serv_name){};
    ~Calculator(){};

    void init(RPCParam *p_param,RPCParam *p_result){

        //converse type from string to expecting type
    }

    void run() {

        // service process code

        // converse type to string

        // save in RPC Result

    }

};

#endif
```

รูปภาพ 4-7:ตัวอย่างการโค้ดในการสร้างเซอร์วิสในระบบ

หลังจากนักพัฒนาได้ทำการพัฒนาเซอร์วิสโค้ดเสร็จแล้ว ก่อนใช้งานนั้น ต้องมีการบันทึกรายละเอียดต่างๆ ที่เกี่ยวข้องกับเซอร์วิสที่ได้สร้างขึ้นมานั้น ได้แก่

- ชื่อเรียกของเซอร์วิส
- ชื่อเต็มของเซอร์วิส
- ข้อความอธิบายเซอร์วิสแบบย่อ
- สถานะในการเปิดใช้งานเมื่อระบบเริ่มต้น

ข้อมูลดังกล่าว จะถูกเก็บไว้ในไฟล์เซอร์วิสรีจิสทรี (Service Registry) ในรูปแบบของภาษาเอ็กซ์เอ็มแอล(XML Format) ไว้สำหรับตั้งค่าต่างๆให้กับเซอร์วิสที่ลงทะเบียนเพื่อเป็นข้อมูลในการเรียกใช้เซอร์วิสต่อไป นอกเหนือจากนี้ นักพัฒนาสามารถเพิ่มฟิลด์ข้อมูลอื่นๆ ที่เป็นลักษณะเฉพาะหรือมีความจำเป็นต้องใช้ในการทำงานของเซอร์วิสได้อีกด้วย

```
<Service type="registry">
  <cal type="key">
    <Name type="string">Calculator</Name>
    <Description type="string">Sample service as calculator</Description>
    <Status type="string">on</Status>
  </cal>
  <address type="key">
    <Name type="string">Address Lists</Name>
    <Description type="string">Sample service as address lists</Description>
    <Status type="string">off</Status>
  </address>
</Service>
```

รูปภาพ 4-8: ตัวอย่างเซอร์วิสรีจิสทรีไฟล์ที่เก็บในรูปแบบเอกสารเอ็กซ์เอ็มแอล

4.3 ผู้ที่เรียกใช้บริการของระบบ

ในลักษณะการใช้งานนี้ ผู้ที่เรียกใช้บริการเซอร์วิสจากระบบฝั่งตัวจะต้องทำการร้องขอรายละเอียดของเซอร์วิสก่อนเพื่อให้ทราบถึงข้อมูลเกี่ยวกับอินเทอร์เฟซในการติดต่อกับเซอร์วิสนั้น จากนั้นสามารถที่จะทำการสร้างแอปพลิเคชันจากไลบรารีต่าง ๆ ขึ้นมาได้ตามสภาพแวดล้อมการใช้งานของผู้ใช้ได้ เช่น อาปาเช่โซบ คอทเน็ตเฟรมเวิร์ก เป็นต้น โดยผู้ใช้งานสามารถนำข้อมูลที่ได้รับกลับมาสร้างเป็นรูปแบบต่างๆ ได้ตามต้องการ อีกทั้งยังสามารถที่จะสร้างอินเทอร์เฟซได้เอง

4.4 ผู้ดูแลระบบการให้บริการของระบบ

ภายในระบบ มีส่วนที่ทำหน้าที่ในการจัดการและควบคุมเซิร์ฟเวอร์ในระบบ โดยผู้ดูแลระบบสามารถพัฒนาแอปพลิเคชันที่ใช้ติดต่อ กับเซิร์ฟเวอร์เมนเจอร์ ในลักษณะของการเรียกใช้เซิร์ฟเวอร์แบบหนึ่ง ซึ่งแอปพลิเคชันนั้น สามารถดัดแปลงรูปแบบได้หลากหลาย อาทิเช่น การเรียกใช้เซิร์ฟเวอร์เมนเจอร์ผ่าน เทลเน็ต (Telnet) ซึ่งสามารถให้แสดงผลในลักษณะแบบของโปรแกรมท๊อป (Top) ที่มีอยู่ในระบบปฏิบัติการลินุกซ์ (Linux) หรือยูนิกซ์ (Unix) หรืออาจเป็นแอปพลิเคชันที่พัฒนาบนแพลตฟอร์มของระบบปฏิบัติการในตระกูลวินโดวส์ (Windows OS Platform) ที่มีจุดเด่นในด้านของการแสดงผลแบบกราฟิกยูสเซอร์อินเตอร์เฟซ (Graphic User Interface)